

# ЛАБОРАТОРНАЯ РАБОТА № 1

## Цветовые модели

### Цель работы

Изучить основные цветовые модели RGB, CMYK, HSV, HLS, XYZ и CIELAB (Lab), принципы перехода между цветовыми пространствами, понятие цветового охвата (gamut) и проблему отображения цветов, не входящих в диапазон sRGB. Освоить разработку интерактивного веб-приложения, позволяющего выбирать и изменять цвет, одновременно отображая его представления в трёх заданных цветовых моделях: RGB, Lab и CMYK.

### Задачи работы

В ходе выполнения лабораторной работы были поставлены следующие задачи:

- изучить назначение и область применения цветовых моделей RGB, CMYK, HSV, HLS, XYZ и Lab;
- рассмотреть преобразования между цветовыми пространствами;
- изучить понятие цветового графика МКО, цветового охвата и выхода цвета за границы отображаемого диапазона;
- реализовать веб-приложение в браузере Safari от 10 версии с одновременным отображением RGB, Lab и CMYK и предупреждение при ограничении цвета до доступного диапазона RGB;
- разместить исходный код в GitHub и подготовить приложение к публикации на статическом хостинге.

### Использованные средства разработки

- Rust — реализация математических функций преобразования цветовых

моделей.

- WebAssembly (WASM) — выполнение Rust-кода в браузере.
- wasm-bindgen 0.2 — организация взаимодействия Rust и JavaScript/TypeScript.
- serde 1 и serde-wasm-bindgen 0.6 — передача структур данных между Rust и JavaScript.
- React — создание интерактивного пользовательского интерфейса.
- TypeScript — типизация данных, результатов преобразований и свойств компонентов.
- Vite — запуск проекта в режиме разработки и формирование production-сборки.
- Git и GitHub — контроль версий и размещение исходного кода.
- Netlify — статический хостинг веб-приложения.

## **Ход выполнения работы**

### **1. Разработка интерфейса**

Было создано веб-приложение Color Lab на React и TypeScript. В интерфейсе сформированы панели RGB, Lab и CMYK, а также отдельная карточка предварительного просмотра выбранного цвета.

Для каждого параметра цвета создан переиспользуемый компонент ColorControl. Он включает подпись параметра, ползунок `input type="range"` для плавного изменения и поле `input type="number"` для точного числового ввода. Таким образом, в каждой из трёх моделей реализованы два способа задания компонентов: ползунок и числовое поле.

### **2. Визуальный выбор цвета**

Для выбора цвета из палитры добавлен элемент `input type="color"`. Он использует системный color picker браузера. Выбранный цвет поступает в HEX-представлении, после чего извлекаются RGB-компоненты. Далее запускается общий механизм пересчёта цветовых моделей.

### **3. Реализация структур в Rust**

В Rust были использованы структуры `Rgb`, `Lab` и `Cmyk`. Каждая структура

содержит компоненты соответствующей модели в виде значений f64. Для передачи результатов вычислений из Rust в TypeScript к структурам был добавлен trait Serialize. Это позволяет преобразовать результат Rust-функции в JavaScript-объект с полями, например:

```
{ l: 42.5, a: 56.2, b: 12.7 }
```

#### 4. Реализация преобразований цветов

В Rust реализованы следующие преобразования

- RGB → Lab;
- Lab → RGB;
- Lab → CMYK;
- CMYK → Lab.

Преобразования Lab → CMYK и CMYK → Lab используют RGB как промежуточное пространство. Это соответствует структуре разработанного Rust-модуля.

#### 5. Подключение Rust к frontend

Для запуска Rust-логики в браузере была создана WASM-обёртка с помощью wasm-bindgen. В TypeScript стали доступны следующие функции:

- rgbToLab(r, g, b);
- labToRgb(l, a, b);
- labToCmyk(l, a, b);
- cmykToLab(c, m, y, k).

Rust-проект был собран в WebAssembly-пакет командой:

```
wasm-pack build --target bundler
```

Полученный пакет был подключён к React-приложению как локальная npm-зависимость.

#### 6. Автоматический пересчёт

Для синхронизации всех моделей были реализованы функции applyRgb, applyLab и applyCmyk.

При изменении RGB выполняется следующая последовательность:

RGB → Lab → CMYK.

При изменении Lab выполняется следующая последовательность:

Lab → RGB;

Lab → CMYK.

При изменении CMYK выполняется следующая последовательность:  
CMYK → Lab → RGB.

После получения результатов React обновляет state всех трёх моделей.

Поэтому на странице одновременно обновляются числовые поля, ползунки, цветовой preview и color picker.

## 7. Проверка диапазонов и округление

В приложении добавлена функция ограничения значений. Она не позволяет передавать значения вне диапазонов:

- RGB: 0–255;
- L: 0–100;
- a и b: от -128 до 127;
- CMYK: 0–100.

Также реализовано округление значений. RGB-каналы отображаются целыми числами, а Lab и CMYK — с точностью до одного знака после запятой.

## 8. Предупреждение о выходе за gamut

При преобразовании Lab → RGB Rust-код возвращает флаг clamped. Он имеет значение true, если при вычислении компоненты RGB вышли за допустимые пределы и были ограничены.

В интерфейсе реализовано ненавязчивое предупреждение:

«Цвет выходит за пределы sRGB. Показан ближайший доступный RGB-цвет.»

Предупреждение отображается только при необходимости и не блокирует дальнейшую работу с приложением.

## Проблемы, возникшие в работе, и способы решения

### 1. Ошибка unresolved import serde.

Причина: библиотека serde не была добавлена в Cargo.toml.

Решение: в зависимости Rust-проекта была добавлена библиотека serde с feature derive.

2. Невозможность прямой передачи Rust-структур в TypeScript.

Причина: JavaScript не может напрямую работать со структурами Rust.

Решение: использованы `Serialize`, `serde-wasm-bindgen` и `JsValue`. Результаты преобразований возвращаются как обычные JavaScript-объекты.

3. Несинхронизированные модели цвета.

Причина: первоначально RGB, Lab и CMYK были представлены независимыми state-переменными. При изменении одной панели другие значения не обновлялись.

Решение: созданы обработчики `applyRgb`, `applyLab` и `applyCmyk`. Они вызывают функции Rust/WASM и обновляют все состояния интерфейса.

4. Выход Lab-цвета за пределы отображаемого RGB-диапазона.

Причина: пространство Lab содержит цвета, которые не могут быть точно отображены в sRGB.

Решение: при преобразовании применяется ограничение RGB-каналов, а пользователю показывается предупреждение о том, что отображён ближайший доступный цвет.

5. Проблема с размещением WASM-пакета в GitHub.

Причина: папка `pkg`, создаваемая `wasm-pack`, не была добавлена в Git-репозиторий.

Решение: WASM-пакет был добавлен принудительно командой `git add -f pkg/`. Это необходимо, поскольку frontend использует локальную зависимость `file:../pkg`.

6. Ошибка размещения на статическом хостинге.

Причина: frontend расположен в подпапке `web`, а хостинг пытался выполнить `npm run build` в корне репозитория, где отсутствует `package.json`.

Решение: в настройках хостинга необходимо указать базовую папку `web`, команду сборки `npm run build` и каталог публикации `dist`.

## Результаты работы

В результате выполнения лабораторной работы создано веб-приложение Color Lab, которое:

- отображает выбранный цвет в RGB, Lab и CMYK;
- позволяет вводить точные значения компонентов через числовые поля;
- позволяет плавно менять компоненты с помощью ползунков;
- позволяет выбирать цвет из палитры браузера;
- автоматически пересчитывает другие модели при изменении любого параметра;
- использует Rust/WASM для выполнения математических преобразований;
- показывает preview текущего RGB-цвета;
- выводит предупреждение при выходе выбранного цвета за пределы sRGB gamut;
- содержит исходный код Rust, React/TypeScript и WASM-обёртки в GitHub-репозитории.

## **Вывод**

Практическим результатом стало создание интерактивного веб-приложения, в котором реализованы все основные способы задания цвета. Для перевода использовал промежуточный формат XYZ и линеаризацию RGB-пространства, так как на практике выяснил, что напрямую перевести невозможно. Для ускорения вычислений использовал компилируемый язык программирования Rust, что позволило проводить конвертацию в реальном времени. Сроки выполнения не были нарушены.